Project Specification: Investigating Linux Scheduling Policies

Linux provides several scheduling policies for managing process and thread execution. The primary policies are:

- `SCHED_OTHER` – Default policy based on the Completely Fair Scheduler (CFS)
- `SCHED_FIFO` – Real-time, First-In-First-Out scheduling
- `SCHED_RR` – Real-time, Round-Robin scheduling
- `SCHED_DEADLINE` – Earliest Deadline First (EDF) scheduling with explicit runtime, deadline, and period parameters

In this project, you will conduct an in-depth study of the Linux scheduler. You will examine how Java thread priorities are mapped to native Linux threads and scheduling priorities. Furthermore, you will design and implement a timing-critical periodic controller using a real-time scheduling policy, with particular emphasis on `SCHED_DEADLINE`. The controller must be pinned to a dedicated CPU core alone.

Finally, you will evaluate system behavior under controlled load conditions using tools such as `app-rt`, `stress-ng`, or a custom-developed stress generator.

Objectives

1. Develop an understanding of Linux scheduling internals, particularly real-time scheduling mechanisms.
2. Investigate the relationship between Java (JVM) threads and native Linux threads.
3. Implement a simple periodic controller in C using that can operate under different scheduling policies, especially `SCHED_DEADLINE` and absolute sleep.
4. Configure CPU affinity by pinning the controller to a specific core and reserving it for controlled experiments.
5. Evaluate real-time performance (latency, jitter, and deadline misses) under varying system loads using `app-rt`, `stress-ng`, and/or a custom load generator:
    - Use absolute timestamps for accurate timing measurements.
    - Optionally, experiment with the `SCHED_FLAG_DL_OVERRUN` flag.

If you require assistance with environment setup, please contact the supervisor.

Good Luck!